

// Jivops

Guía completa — 30 lecciones

Arquitectura · Validación · Mutación y generación · Seguridad de
imágenes · Operación a escala · Madurez y gobernanza

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es Kyverno y comparación con OPA/Gatekeeper
- Lección 02 — Arquitectura de Kyverno — admission controller, background scanning
- Lección 03 — Instalación de Kyverno (Helm)
- Lección 04 — Tipos de políticas — validate, mutate, generate, verifyImages
- Lección 05 — ClusterPolicy vs Policy (namespaced)

Módulo 2: Políticas de validación

- Lección 06 — Reglas de validación — sintaxis básica
- Lección 07 — Validación de labels y annotations obligatorios
- Lección 08 — Bloqueo de imágenes con tag 'latest'
- Lección 09 — Exigir resource limits en Pods
- Lección 10 — Validación con expresiones CEL

Módulo 3: Mutación y generación

- Lección 11 — Políticas de mutación — inyección automática de configuración
- Lección 12 — Generación automática de recursos (NetworkPolicy por namespace)
- Lección 13 — Sincronización de recursos generados (generateExisting, sync)
- Lección 14 — Casos de uso de mutación — defaults seguros
- Lección 15 — Orden de ejecución — validate vs mutate vs generate

Módulo 4: Seguridad de imágenes

- Lección 16 — Verificación de firmas de imágenes (Cosign/Sigstore)
- Lección 17 — Verificación de SBOM y atestaciones
- Lección 18 — Políticas de Pod Security Standards con Kyverno
- Lección 19 — Bloqueo de registries no autorizados
- Lección 20 — Integración con Notation/Notary

Módulo 5: Operación a escala

- Lección 21 — Modos de política — Enforce vs Audit
- Lección 22 — Background scanning de recursos existentes
- Lección 23 — Exclusiones y excepciones (PolicyException)
- Lección 24 — Rendimiento del admission webhook
- Lección 25 — Reportes de políticas (PolicyReport)

Módulo 6: Madurez y gobernanza

Lección 26 — Kyverno como reemplazo de OPA/Gatekeeper — migración

Lección 27 — Gestión de políticas a escala (múltiples clusters con GitOps)

Lección 28 — Testing de políticas antes de producción

Lección 29 — RBAC y gobernanza de quién puede modificar políticas

Lección 30 — Proyecto final: framework de políticas completo con Kyverno listo para producción

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es Kyverno y comparación con OPA/Gatekeeper

Kyverno usa YAML nativo de Kubernetes, reduciendo la barrera de entrada frente a aprender un lenguaje separado como Rego de OPA/Gatekeeper.

Lección 02 — Arquitectura de Kyverno — admission controller, background scanning

El background scanning detecta recursos que ya existían antes de crear una política, complementando lo que el admission control solo cubre para recursos nuevos.

Lección 03 — Instalación de Kyverno (Helm)

Probar primero en modo audit permite ver qué recursos violarían la política sin bloquear nada, antes de pasar a modo enforce.

```
helm install kyverno kyverno/kyverno
```

Lección 04 — Tipos de políticas — validate, mutate, generate, verifyImages

verifyImages verifica específicamente la firma criptográfica de una imagen, un tipo de validación distinto a solo revisar campos del manifiesto.

Lección 05 — ClusterPolicy vs Policy (namespaced)

Una ClusterPolicy aplica consistentemente a todos los namespaces relevantes, evitando duplicar la misma política en cada uno por separado.

```
kind: ClusterPolicy
```

Módulo 2: Políticas de validación

Lección 06 — Reglas de validación — sintaxis básica

Un mensaje personalizado claro explica al desarrollador qué regla violó y cómo corregirlo, en vez de un error genérico confuso.

Lección 07 — Validación de labels y annotations obligatorios

Automatizar la exigencia con Kyverno garantiza cumplimiento real, sin depender de que cada desarrollador recuerde seguir la buena práctica voluntariamente.

```
required: [team]
```

Lección 08 — Bloqueo de imágenes con tag 'latest'

Con un tag específico se sabe exactamente qué versión corre y a cuál volver, mientras 'latest' podría apuntar a algo distinto según cuándo se construyó.

```
image: "!:latest"
```

Lección 09 — Exigir resource limits en Pods

Sin límite de memoria definido, un Pod podría consumir recursos desproporcionados del nodo compartido, afectando a los Pods vecinos.

Lección 10 — Validación con expresiones CEL

CEL permite lógica más expresiva, como comparar 'requests.cpu' contra 'limits.cpu' del mismo contenedor, algo difícil con un pattern estático.

```
object.spec.replicas <= 10
```

Módulo 3: Mutación y generación

Lección 11 — Políticas de mutación — inyección automática de configuración

Inyectar configuración estándar vía mutación garantiza su aplicación consistente, sin depender de que cada equipo la incluya manualmente.

Lección 12 — Generación automática de recursos (NetworkPolicy por namespace)

Generar una NetworkPolicy base en cada namespace nuevo evita que alguno quede accidentalmente sin protección de red por olvido.

```
kind: NetworkPolicy
```

Lección 13 — Sincronización de recursos generados (generateExisting, sync)

Sin sync activado, un cambio posterior en la política fuente no se reflejaría automáticamente en los recursos ya generados previamente.

```
synchronize: true
```

Lección 14 — Casos de uso de mutación — defaults seguros

Aplicar un default seguro vía mutación es más amigable que bloquear: corrige automáticamente en vez de rechazar el despliegue.

```
runAsNonRoot: true
```

Lección 15 — Orden de ejecución — validate vs mutate vs generate

mutate se ejecuta antes que validate: si ocurriera al revés, podría rechazarse un recurso que la mutación habría corregido después.

Módulo 4: Seguridad de imágenes

Lección 16 — Verificación de firmas de imágenes (Cosign/Sigstore)

Bloquear imágenes sin firma válida evita que una imagen maliciosa o no autorizada llegue a ejecutarse en el cluster.

Lección 17 — Verificación de SBOM y atestaciones

Una atestación de escaneo exitoso confirma que la imagen cumple una condición adicional, más allá de solo verificar quién la firmó.

Lección 18 — Políticas de Pod Security Standards con Kyverno

Combinar Pod Security Standards con políticas propias en el mismo motor de Kyverno mantiene toda la gobernanza en un sistema coherente.

Lección 19 — Bloqueo de registries no autorizados

Restringir el origen de las imágenes a un registry corporativo garantiza que solo se despliegue lo que pasó por el proceso de auditoría interno.

```
image: "miregistro.empresa.com/*"
```

Lección 20 — Integración con Notation/Notary

Notation sigue el estándar Notary v2, una alternativa a Cosign para organizaciones ya alineadas con ese ecosistema específico.

Módulo 5: Operación a escala

Lección 21 — Modos de política — Enforce vs Audit

Empezar en modo Audit descubre cuántos recursos existentes violarían la política, sin bloquear nada, antes de pasar a Enforce.

```
validationFailureAction: Audit
```

Lección 22 — Background scanning de recursos existentes

El background scanning da visibilidad inmediata del cumplimiento real del cluster, clave antes de decidir activar el modo enforce.

Lección 23 — Exclusiones y excepciones (PolicyException)

Una excepción acotada y documentada exime solo el caso específico legítimo, sin perder la protección de la política para el resto del cluster.

```
kind: PolicyException
```

Lección 24 — Rendimiento del admission webhook

Consolidar reglas relacionadas en menos políticas reduce el overhead de evaluar muchas políticas pequeñas por separado.

Lección 25 — Reportes de políticas (PolicyReport)

Un dashboard consolidado de PolicyReports ayuda a priorizar qué violaciones corregir primero, sin revisar reporte por reporte individualmente.

```
kubectl get policyreport
```


Módulo 6: Madurez y gobernanza

Lección 26 — Kyverno como reemplazo de OPA/Gatekeeper — migración

Migrar y validar una política a la vez en modo Audit reduce el riesgo frente a traducir y activar todas las políticas de golpe.

Lección 27 — Gestión de políticas a escala (múltiples clusters con GitOps)

El historial de Git permite rastrear qué commit modificó una política específica, facilitando entender su evolución con el tiempo.

Lección 28 — Testing de políticas antes de producción

'kyverno test' valida la lógica localmente contra casos de prueba, antes de arriesgar aplicarla directamente en un cluster real.

```
kyverno test ./mi-politica/
```

Lección 29 — RBAC y gobernanza de quién puede modificar políticas

Restringir quién puede crear una ClusterPolicy limita el riesgo de un recurso que puede debilitar la seguridad de todo el cluster.

Lección 30 — Proyecto final: framework de políticas completo con Kyverno listo para producción

Se integra todo lo anterior: políticas validate/mutate/generate bien probadas, modo audit antes de enforce, verificación de firmas de imágenes, PolicyExceptions documentadas, RBAC restringido y gestión GitOps multi-cluster — el nivel esperado de un framework de políticas con Kyverno listo para producción real.